

## **EXPERIMENT 3**

**Aim:** Create a Cryptocurrency using Python and perform mining in the Blockchain created.

**Theory:**

### **1. Blockchain Overview**

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

### **2. Mining**

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.  
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

### **3. Multi-Node Blockchain Network**

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

### **4. Consensus Mechanism**

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

### **5. Transactions & Mining Reward**

Each transaction has:

- Sender

- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

## 6. Chain Replacement

When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

## Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints  
`pip install Flask`
- **Requests** – For HTTP communication between nodes  
`pip install requests==2.18.4`
- **Postman** – For testing API requests
- Python Standard Libraries:
  - `datetime`
  - `jsonify`
  - `hashlib`
  - `uuid4`
  - `urlparse`
  - `request`

## Procedure and Output:

1. Download `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py`.
2. Install required packages.
3. Run each node in separate terminals.

Problems Output Debug Console Terminal Ports

```
• PS C:\Lab_3_Blockchain> python -m venv venv
• PS C:\Lab_3_Blockchain> venv\Scripts\activate
• (venv) PS C:\Lab_3_Blockchain> pip install Flask==0.12.2
Collecting Flask==0.12.2
  Downloading Flask-0.12.2-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting Jinja2>=2.4 (from Flask==0.12.2)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting Werkzeug>=0.7 (from Flask==0.12.2)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting click>=2.0 (from Flask==0.12.2)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=0.21 (from Flask==0.12.2)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=2.4->Flask==0.12.2)
  Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
```

```
-----
○ (venv) PS C:\Lab_3_Blockchain> python hadcoin_node_5001.py
* Serving Flask app 'hadcoin_node_5001'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.0.102:5001
Press CTRL+C to quit
█
```

```
• PS C:\Lab_3_Blockchain> & c:/Lab_3_Blockchain/venv/Scripts/Activate.ps1
○ (venv) PS C:\Lab_3_Blockchain> python hadcoin_node_5002.py
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.0.102:5002
Press CTRL+C to quit
█
```

Problems Output Debug Console Terminal Ports

```
• PS C:\Lab_3_Blockchain> & c:/Lab_3_Blockchain/venv/Scripts/Activate.ps1
○ (venv) PS C:\Lab_3_Blockchain> python hadcoin_node_5003.py
* Serving Flask app 'hadcoin_node_5003'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.0.102:5003
Press CTRL+C to quit
█
```

4. Connect nodes using Postman – POST [/connect\\_node](#).

http://127.0.0.1:5001/connect\_node

POST http://127.0.0.1:5001/connect\_node Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5002",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED 79 ms 325 B

{ JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5003"
6   ]
7 }
8
```

http://127.0.0.1:5002/connect\_node

POST http://127.0.0.1:5002/connect\_node Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED 127 ms 325 B

{ JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5003",
5     "127.0.0.1:5001"
6   ]
7 }
8
```

HTTP [http://127.0.0.1:5003/connect\\_node](http://127.0.0.1:5003/connect_node) Save Share

POST [http://127.0.0.1:5003/connect\\_node](http://127.0.0.1:5003/connect_node) Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐ Schema Beautify

```
1 {
2   |   "nodes": [
3   |     "http://127.0.0.1:5001",
4   |     "http://127.0.0.1:5002"
5   |   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED • 139 ms • 325 B •

{ } JSON Preview Visualize

```
1 {
2   |   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   |   "total_nodes": [
4   |     "127.0.0.1:5001",
5   |     "127.0.0.1:5002"
6   |   ]
7 }
8
```

## 5. Add transactions – POST

HTTP [http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction) Save Share

POST [http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction) Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐ Schema Beautify

```
1 {
2   |   "sender": "Hitika",
3   |   "receiver": "Danice",
4   |   "amount": 51
5 }
```

Body Cookies Headers (5) Test Results 201 CREATED • 196 ms • 226 B •

{ } JSON Preview Visualize

```
1 {
2   |   "message": "This transaction will be added to Block 2"
3 }
4
```

HTTP [http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction) Save Share

**POST** [http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction) Send

Docs Params Authorization Headers (8) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "sender": "Hitika",
3   "receiver": "Danice",
4   "amount": 52
5 }
```

**Body** Cookies Headers (5) Test Results 201 CREATED • 129 ms • 226 B 🌐 ⋮

**{}** **JSON** Preview Visualize 🔍 📄 🔗

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
4
```

HTTP [http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction) Save Share

**POST** [http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction) Send

Docs Params Authorization Headers (8) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "sender": "Hitika",
3   "receiver": "Danice",
4   "amount": 53
5 }
```

**Body** Cookies Headers (5) Test Results 201 CREATED • 107 ms • 226 B 🌐 ⋮

**{}** **JSON** Preview Visualize 🔍 📄 🔗

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
4
```

HTTP [http://127.0.0.1:5001/mine\\_block](http://127.0.0.1:5001/mine_block)

GET [http://127.0.0.1:5001/mine\\_block](http://127.0.0.1:5001/mine_block)

Docs Params Authorization Headers (8) Body ● Scripts Settings

Body Cookies Headers (5) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1  {
2    "index": 2,
3    "message": "Congratulations, you just mined a block!",
4    "previous_hash": "085f1f5ee5c2b31f5684aab0e64f2063da94ac31ef22a1df5e9b960c37eb4a57",
5    "proof": 533,
6    "timestamp": "2026-09-16 23:24:27.328514",
7    "transactions": [
8      {
9        "amount": 51,
10       "receiver": "Danice",
11       "sender": "Hitika"
12     },
13     {
14       "amount": 52,
15       "receiver": "Danice",
16       "sender": "Hitika"
17     },
18     {
19       "amount": 53,
20       "receiver": "Danice",
21       "sender": "Hitika"
22     },
23     {
24       "amount": 1,
25       "receiver": "Richard",
26       "sender": "bb78e1a1e64a4881aa9878d470c74592"
27     }
28   ]
29 }
```

6. Each node must have **different chain lengths**

HTTP http://127.0.0.1:5001/get\_chain

GET http://127.0.0.1:5001/get\_chain

Docs Params Authorization Headers (8) Body ● Scripts Settings

Body Cookies Headers (5) Test Results ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
15     "transactions": [  
16         {  
17             "amount": 51,  
18             "receiver": "Danice",  
19             "sender": "Hitika"  
20         },  
21         {  
22             "amount": 52,  
23             "receiver": "Danice",  
24             "sender": "Hitika"  
25         },  
26         {  
27             "amount": 53,  
28             "receiver": "Danice",  
29             "sender": "Hitika"  
30         },  
31         {  
32             "amount": 1,  
33             "receiver": "Richard",  
34             "sender": "bb78e1a1e64a4881aa9878d470c74592"  
35         }  
36     ],  
37     }  
38 ],  
39     "length": 2  
40 }  
41
```

HTTP [http://127.0.0.1:5002/get\\_chain](http://127.0.0.1:5002/get_chain)

GET [http://127.0.0.1:5002/get\\_chain](http://127.0.0.1:5002/get_chain)

Docs Params Authorization Headers (8) Body • Scripts Settings

Body Cookies Headers (5) Test Results ↺

{ } JSON ▾ ▷ Preview 🖼 Visualize ▾

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-16 21:57:58.075726",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
13
```

HTTP [http://127.0.0.1:5003/get\\_chain](http://127.0.0.1:5003/get_chain)

GET [http://127.0.0.1:5003/get\\_chain](http://127.0.0.1:5003/get_chain)

Docs Params Authorization Headers (8) Body • Scripts Settings

Body Cookies Headers (5) Test Results ↺

{ } JSON ▾ ▷ Preview 🖼 Visualize ▾

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-16 21:58:27.465125",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
13
```

## 7. Replacing the Chain Using Longest Chain Rule – Node 5002 and 5003



http://127.0.0.1:5002/replace\_chain

GET



http://127.0.0.1:5002/replace\_chain

Docs

Params

Authorization

Headers (8)

Body •

Scripts

Settings

Body

Cookies

Headers (5)

Test Results



20

{ } JSON



Preview



Visualize

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-09-16 21:56:05.028803",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash": "085f1f5ee5c2b31f5684aab0e64f2063da94ac31ef22a1df5e9b960c37eb4a57",
14      "proof": 533,
15      "timestamp": "2026-09-16 23:24:27.328514",
16      "transactions": [
17        {
18          "amount": 51,
19          "receiver": "Danice",
20          "sender": "Hitika"
21        },
22        {
23          "amount": 52,
24          "receiver": "Danice",
25          "sender": "Hitika"
26        }
27      ]
28    }
29  ]
30 }
```

HTTP [http://127.0.0.1:5002/get\\_chain](http://127.0.0.1:5002/get_chain)

GET [http://127.0.0.1:5002/get\\_chain](http://127.0.0.1:5002/get_chain)

Docs Params Authorization Headers (8) Body • Scripts Settings

Body Cookies Headers (5) Test Results ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
15     "transactions": [  
16         {  
17             "amount": 51,  
18             "receiver": "Danice",  
19             "sender": "Hitika"  
20         },  
21         {  
22             "amount": 52,  
23             "receiver": "Danice",  
24             "sender": "Hitika"  
25         },  
26         {  
27             "amount": 53,  
28             "receiver": "Danice",  
29             "sender": "Hitika"  
30         },  
31         {  
32             "amount": 1,  
33             "receiver": "Richard",  
34             "sender": "bb78e1a1e64a4881aa9878d470c74592"  
35         }  
36     ],  
37     }  
38 ],  
39     "length": 2  
40 }  
41
```

http://127.0.0.1:5003/replace\_chain

GET http://127.0.0.1:5003/replace\_chain

Docs Params Authorization Headers (8) Body Scripts Settings

Body Cookies Headers (5) Test Results

{ } JSON Preview Visualize

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-09-16 21:56:05.028803",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash": "085f1f5ee5c2b31f5684aab0e64f2063da94ac31ef22a1df5e9b960c37eb4a57",
14      "proof": 533,
15      "timestamp": "2026-09-16 23:24:27.328514",
16      "transactions": [
17        {
18          "amount": 51,
19          "receiver": "Danice",
20          "sender": "Hitika"
21        },
22        {
23          "amount": 52,
24          "receiver": "Danice",
25          "sender": "Hitika"
26        }
27      ]
28    }
29  ]
30 }
```

Postbot Runner

## 8. Validation of Blockchain

http://127.0.0.1:5001/is\_valid

GET http://127.0.0.1:5001/is\_valid

Docs Params Authorization Headers (8) Body Scripts Settings

Body Cookies Headers (5) Test Results

{ } JSON Preview Visualize

```
1 {
2   "message": "All good. The Blockchain is valid."
3 }
4
```

200 OK • 195 ms • 214 B •

HTTP [http://127.0.0.1:5002/is\\_valid](http://127.0.0.1:5002/is_valid)

GET [http://127.0.0.1:5002/is\\_valid](http://127.0.0.1:5002/is_valid)

Docs Params Authorization Headers (8) Body ● Scripts S

Body Cookies Headers (5) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {  
2   "message": "All good. The Blockchain is valid."  
3 }  
4
```

HTTP [http://127.0.0.1:5003/is\\_valid](http://127.0.0.1:5003/is_valid)

GET [http://127.0.0.1:5003/is\\_valid](http://127.0.0.1:5003/is_valid)

Docs Params Authorization Headers (8) Body ● Scripts

Body Cookies Headers (5) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {  
2   "message": "All good. The Blockchain is valid."  
3 }  
4
```

## 8. Removing Transactions After Adding Them to a Block

```

19 class Blockchain:
20
21
22
23
24
25
26
27 def create_block(self, proof, previous_hash):
28     block = {'index': len(self.chain) + 1,
29             'timestamp': str(datetime.datetime.now()),
30             'proof': proof,
31             'previous_hash': previous_hash,
32             'transactions': self.transactions}
33     self.transactions = []
34     self.chain.append(block)
35     return block
36
37 def get_previous_block(self):
38     return self.chain[-1]
39
40 def proof_of_work(self, previous_proof):
41     new_proof = 1
42     check_proof = False
43     while check_proof is False:
44         hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
45         if hash_operation[:4] == '0000':
46             check_proof = True
47         else:
48             new_proof += 1
49     return new_proof
50
51 def hash(self, block):
52     encoded_block = json.dumps(block, sort_keys = True).encode()
53     return hashlib.sha256(encoded_block).hexdigest()
54
55 def is_chain_valid(self, chain):

```

## 9.Adding a New Transaction After Mining


[http://127.0.0.1:5001/add\\_transaction](http://127.0.0.1:5001/add_transaction)

POST

▼

http://127.0.0.1:5001/add\_transaction

☰ Docs

Params

Authorization

Headers (8)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

```

1  {
2      "sender": "Ed",
3      "receiver": "Hitika",
4      "amount": 500
5  }

```

Body

Cookies

Headers (5)

Test Results

↺

{}

JSON ▼

▶ Preview

🖼 Visualize

▼

```

1  {
2      "message": "This transaction will be added to Block 3"
3  }
4

```

## 10. Mining the Second Block

HTTP [http://127.0.0.1:5001/mine\\_block](http://127.0.0.1:5001/mine_block)

GET [http://127.0.0.1:5001/mine\\_block](http://127.0.0.1:5001/mine_block)

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ☐

1 5

Body Cookies Headers (5) Test Results

{ } JSON ☐ Preview Visualize ☐

```
1 {
2   "index": 3,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "44717517229fdaf41f32ece8b659420213e96448607955fcd237765e7cf94872",
5   "proof": 45293,
6   "timestamp": "2026-09-17 00:30:06.316777",
7   "transactions": [
8     {
9       "amount": 500,
10      "receiver": "Hitika",
11      "sender": "Ed"
12    },
13    {
14      "amount": 1,
15      "receiver": "Richard",
16      "sender": "bb78e1a1e64a4881aa9878d470c74592"
17    }
18  ]
19 }
20
```

## 11. Final blockchain synchronization

HTTP [http://127.0.0.1:5002/replace\\_chain](http://127.0.0.1:5002/replace_chain)

GET [http://127.0.0.1:5002/replace\\_chain](http://127.0.0.1:5002/replace_chain)

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ☐

1 5

Body Cookies Headers (5) Test Results

{ } JSON ☐ Preview Visualize ☐

```
37   }
38   },
39   {
40     "index": 3,
41     "previous_hash": "44717517229fdaf41f32ece8b659420213e96448607955fcd237765e7cf94872",
42     "proof": 45293,
43     "timestamp": "2026-09-17 00:30:06.316777",
44     "transactions": [
45       {
46         "amount": 500,
47         "receiver": "Hitika",
48         "sender": "Ed"
49       },
50       {
51         "amount": 1,
52         "receiver": "Richard",
53         "sender": "bb78e1a1e64a4881aa9878d470c74592"
54       }
55     ]
56   }
57 ]
58 }
59
```

```
37     ]
38   },
39   {
40     "index": 3,
41     "previous_hash": "44717517229fdaf41f32ece8b659420213e96448607955fcd237765e7cf94872",
42     "proof": 45293,
43     "timestamp": "2026-09-17 00:30:06.316777",
44     "transactions": [
45       {
46         "amount": 500,
47         "receiver": "Hitika",
48         "sender": "Ed"
49       },
50       {
51         "amount": 1,
52         "receiver": "Richard",
53         "sender": "bb78e1a1e64a4881aa9878d476c74592"
54       }
55     ]
56   }
57 ]
58 }
59 }
```

## Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.